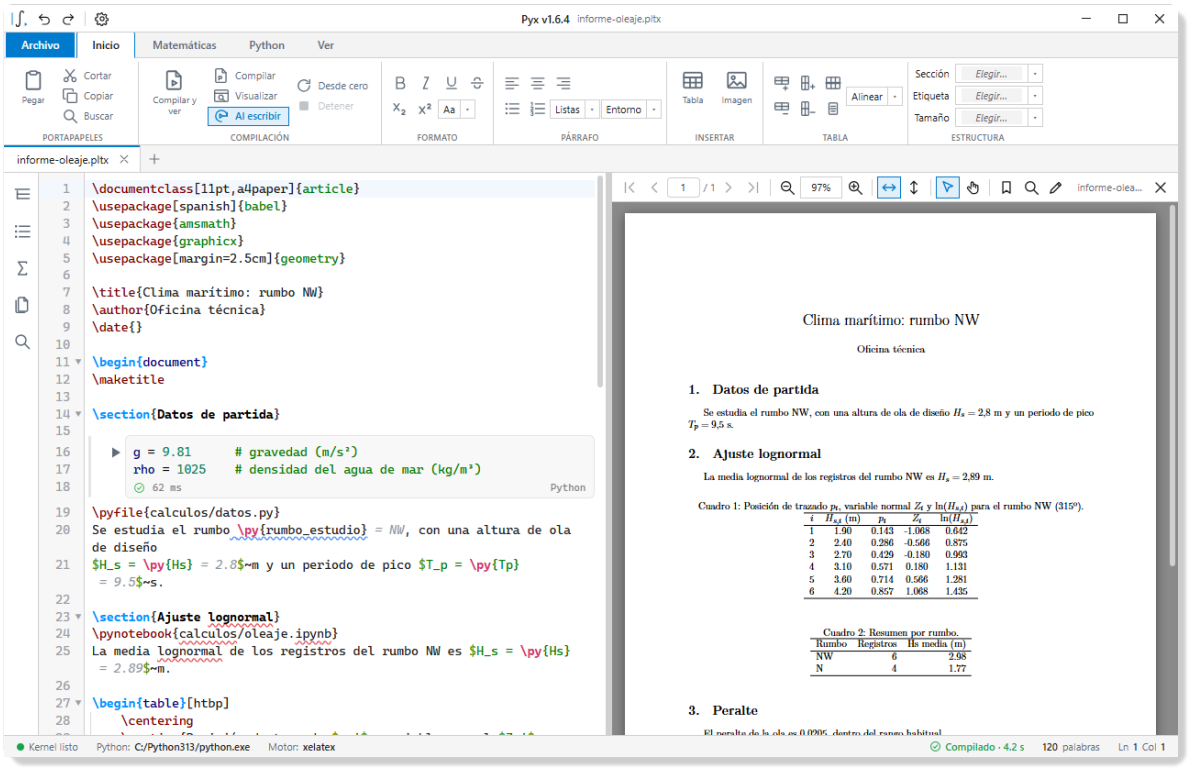
Pyx

Manual de usuario

LaTeX y Python en un mismo documento:
escribe el informe, calcula en él y obtén el PDF.



Versión 1.6.4
octubre de 2026

Windows · macOS · Linux
Escrito y compilado con Pyx

Pyx — Manual de usuario

Versión 1.6.4 · 25 de octubre de 2026

Este manual se ha escrito con Pyx y se ha compilado con Pyx: es un documento `.pltx` con celdas de Python que se ejecutaron al compilarlo, con Python 3.13.7 y NumPy 2.3.4. La figura 7.1 y los valores numéricos del capítulo 7 los calculó una de esas celdas; ninguno se escribió a mano.

Pyx es software libre, publicado con la licencia MIT. Código fuente, instaladores y registro de cambios: `https://github.com/LorGIOO/Pyx`.

Las capturas de pantalla se tomaron en Windows; en macOS y Linux la aplicación es idéntica, salvo los botones de la ventana y las rutas de los archivos.

Índice general

1 Bienvenida **3**

1.1 Qué hace Pyx . . . 3

1.2 Novedades de la versión 1.6.4 . . . 4

1.3 Cómo leer este manual . . . 4

2 Instalación y primer arranque **5**

2.1 Requisitos . . . 5

2.2 Instalar Pyx . . . 5

2.3 Preparar Python y LaTeX con un clic . . . 5

2.4 Elegir el intérprete de Python . . . 6

2.5 La pantalla de inicio . . . 7

3 Recorrido por la interfaz **8**

3.1 La cinta . . . 9

3.2 El panel lateral . . . 9

3.3 Temas . . . 9

4 Documentos y proyectos **11**

4.1 Crear un documento . . . 11

4.2 El formato `.pltx` . . . 11

4.3 Abrir, guardar y cerrar . . . 12

4.4 Proyectos de varios archivos . . . 12

5 Escribir LaTeX **13**

5.1 Autocompletado . . . 13

5.2 La cinta de formato . . . 14

5.3 Tablas . . . 14

5.4 Imágenes y símbolos . . . 15

5.5 Fórmulas: vista previa al pasar el ratón . . . 15

5.6 Navegar por el documento . . . 16

5.7 Edición inteligente . . . 16

5.8 Comprobaciones mientras escribes . . . 16

5.9 Ortografía y gramática . . . 16

5.10 Buscar y reemplazar 17

6 Celdas de Python **19**

6.1 Crear y ejecutar celdas 19
6.2 Las salidas 19
6.3 Errores 20
6.4 Autocompletado y ayuda de Python 21
6.5 El kernel 22

7 Del cálculo al documento **23**

7.1 Un valor en el texto: `\py{}` 23
7.2 Un texto u otro: `\pyif` 24
7.3 Tablas y figuras completas: el módulo `pyx` 24
7.4 Un ejemplo completo 24
7.5 El desarrollo del cálculo: handcalcs 25

8 Cuadernos de Jupyter y archivos de Python **26**

8.1 Qué se puede usar 26
8.2 Sin volver a ejecutarlos 26
8.3 Archivos `.py` y sus celdas 27
8.4 Varios archivos y variables con el mismo nombre 27

9 Ejecución reactiva **28**

9.1 Qué se ejecuta 28
9.2 Celdas desactualizadas 28
9.3 Mayúsculas+Intro, reactivo 29
9.4 Garantías 29

10 Compilar **31**

10.1 Las formas de compilar 31
10.2 Rápido: el motor precalentado 31
10.3 Motores 31
10.4 Bibliografía, índices y glosarios 32
10.5 El registro y los problemas 32

11 El visor de PDF **34**

11.1 La barra del visor 34

11.2 Del código al PDF y del PDF al código (SyncTeX) 34

11.3 Marcadores 35

12 Macros en Python **36**

13 Configuración **38**

14 Atajos de teclado **40**

15 Solución de problemas **42**

16 Glosario **44**

Créditos **45**

1

Bienvenida

Pyx es un editor de documentos LaTeX con celdas de Python dentro. En el mismo archivo escribes el texto del informe, haces los cálculos y colocas sus resultados en el texto; al compilar, Pyx ejecuta las celdas, sustituye cada valor en su sitio y genera el PDF. Si cambias un dato, todo lo que depende de él se actualiza solo: el número del párrafo, la tabla, la gráfica y la conclusión.

Está pensado para quien escribe documentos técnicos con números dentro (memorias de cálculo, informes de laboratorio, trabajos académicos, anejos de proyecto) y está cansado de copiar resultados de una hoja de cálculo o de Jupyter al documento, y de descubrir después que el PDF y los cálculos ya no coinciden.

1.1 Qué hace Pyx

Un editor de LaTeX completo. Resaltado, autocompletado de más de 300 comandos y entornos, comprobación de errores mientras escribes, ortografía y gramática, plegado, marcadores y búsqueda en todo el proyecto: lo que esperas de TeXstudio.

Celdas de Python, como en Jupyter. Bloques de código dentro del documento, con su salida debajo: texto, tablas, gráficas, fórmulas. Autocompletado real con jedi y ayuda de cada función.

Tus cuadernos y scripts, sin repetir el cálculo. Un cuaderno de Jupyter (`.ipynb`) o un archivo `.py` se vinculan al documento y sus variables se usan igual que las de una celda. Si el cuaderno está abierto en VS Code o Jupyter, Pyx toma sus valores de allí al momento, sin ejecutar nada.

Un puente del cálculo al texto. `\py{expresión}` escribe en el PDF el valor de una expresión de Python, y `\pyif` elige un texto u otro según el resultado.

Un visor de PDF integrado. El PDF se ve al lado del código, con salto en los dos sentidos (Ctrl+clic), búsqueda, marcadores y anotaciones.

Rapidez. Pyx recalcula solo las celdas afectadas por un cambio y mantiene LaTeX con el preámbulo ya cargado, así que el PDF se actualiza en décimas de segundo mientras escribes.

1.2 Novedades de la versión 1.6.4

- **Cuadernos de Jupyter y archivos `.py` vinculados** con `\pynotebook{…}` y `\pyfile{…}`: sus variables, funciones y tablas se usan en el documento sin volver a ejecutarlos (capítulo 8).
- **El valor del archivo más próximo**: si dos archivos (o dos celdas) dan valor a la misma variable, cada `\py{}` muestra el del que tiene encima.
- `\py{df}` con una tabla de pandas escribe una **tabla de LaTeX**.
- Dentro de `\py{` el **autocompletado** lista todas las variables disponibles: las de las celdas y las de los cuadernos y archivos vinculados.
- **Pegar** es más limpio: el código con sangría ya no se convierte en tabla, se quitan los caracteres invisibles y, al pegar una celda copiada, recibe un identificador propio.

Y lo que trajo la versión 1.6: la **ejecución reactiva** (capítulo 9); el **autocompletado de Python con jedi**, con la firma de cada función; la preparación de Python y LaTeX **sin instalaciones previas** (sección 2.3); **buscar en todo el proyecto**; **pegar tablas de Excel**; el **autocompletado de LaTeX** con más de 300 comandos; las **macros en Python**, los **glosarios**, los **marcadores del PDF**, el **historial del cursor** y la gramática con **LanguageTool**.

1.3 Cómo leer este manual

Los capítulos 2 a 4 explican cómo instalar Pyx, cómo es la ventana y cómo se trabaja con documentos. Los capítulos 5 a 9 recorren lo que haces a diario: escribir LaTeX, programar en celdas, usar tus cuadernos de Jupyter y archivos de Python, y llevar los resultados al texto. A partir del capítulo 10 se explican la compilación, el visor de PDF y las herramientas de apoyo. Al final hay una tabla con todos los atajos de teclado (capítulo 14) y un capítulo de solución de problemas.

A lo largo del texto encontrarás tres tipos de recuadro:

Consejo Una forma más rápida o más cómoda de hacer algo.

Nota Un detalle que conviene conocer para entender lo que hace Pyx.

Atención Algo que puede dar un resultado inesperado si no se tiene en cuenta.

Las teclas aparecen así: `Ctrl`+`S`. En macOS, `Ctrl` es `Cmd`. Los botones de la aplicación aparecen así: Compilar y ver.

2

Instalación y primer arranque

2.1 Requisitos

- **Windows** 10 u 11 (64 bits), **macOS** 11 o posterior (Intel o Apple Silicon) o **Linux** de 64 bits (Ubuntu 22.04 o equivalente).
- 4 GB de memoria y unos 600 MB de disco si Pyx prepara su propio Python, o 200 MB si usa uno que ya tengas.
- Conexión a Internet solo la primera vez, si Pyx tiene que preparar Python o LaTeX.

No necesitas instalar nada más. Si ya tienes Python (Anaconda, python.org, Microsoft Store…) o una distribución de LaTeX (MiKTeX, TeX Live, MacTeX), Pyx los encuentra y los usa. Si no, te ofrece prepararlos.

2.2 Instalar Pyx

Descarga el instalador de tu sistema desde la página de versiones (`https://github.com/LorGIOO/Pyx/releases`). Junto a los instaladores de cada versión está también este manual, en PDF, de esa misma versión.

Windows: `Pyx_1.6.4_x64-setup.exe`. Ábrelo y sigue el asistente. Al terminar, los archivos `.pltx` (y, si quieres, los `.tex`) se abren con Pyx al hacer doble clic.

macOS: `Pyx_1.6.4_universal.dmg`, válido para Mac con Intel y con Apple Silicon. Arrastra Pyx a Aplicaciones. La primera vez, abre la aplicación con clic derecho → **Abrir**.

Linux: `.deb` (Ubuntu, Debian), `.rpm` (Fedora) o `.AppImage` (cualquier distribución: dale permiso de ejecución y ábrelo).

2.3 Preparar Python y LaTeX con un clic

Al arrancar, Pyx comprueba si hay un intérprete de Python y un motor de LaTeX. Si falta alguno, aparece el diálogo **Preparar Pyx** (figura 2.1).

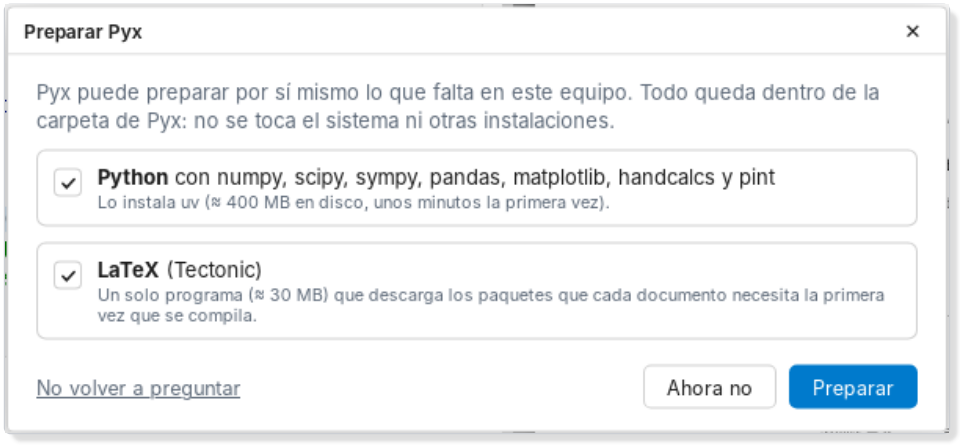


Figura 2.1: El diálogo «Preparar Pyx» aparece cuando falta Python o LaTeX.

Python. Pyx descarga *uv*, un instalador de Python muy rápido, y con él un Python 3.12 independiente con las librerías que usa un informe de cálculo: NumPy, SciPy, SymPy, pandas, Matplotlib, handcalcs, Pint, openpyxl, IPython y jedi. Tarda entre uno y cinco minutos según la conexión, y ocupa unos 400 MB.

LaTeX (Tectonic). Un único programa de unos 30 MB que, la primera vez que compila un documento, descarga solo los paquetes de LaTeX que ese documento necesita.

Pulsa Preparar. Verás en directo lo que se descarga e instala. Al terminar, Pyx pasa a usar ese Python y ese LaTeX.

> **Dónde se instala** Todo queda dentro de la carpeta de datos de Pyx: %LOCALAPPDATA%\Pyx en Windows, ~/Library/Application Support/Pyx en macOS y ~/.local/share/pyx en Linux. No se modifica el PATH ni se toca ninguna otra instalación. Para desinstalarlo, basta con borrar esa carpeta.

Si ya tienes Python y LaTeX, el diálogo no aparece. Si lo cierras con Ahora no, puedes abrirlo más tarde desde **Configuración** → **Celdas Python** → **Python de Pyx** o **Configuración** → **Compilación** → **Descargar Tectonic**. No volver a preguntar evita que aparezca al arrancar.

2.4 Elegir el intérprete de Python

Pyx usa, por este orden:

1. el intérprete que hayas elegido tú (**Python** → **Intérprete**, o **Configuración** → **Celdas Python** → **Ruta de Python**);
2. el Python que haya preparado el propio Pyx;
3. un Python del sistema que tenga NumPy;
4. cualquier otro Python que funcione.

La barra de estado indica siempre el intérprete en uso. El desplegable **Python** de la pestaña **Python** de la cinta lista todos los que Pyx encuentra en el equipo (junto a cada uno se indica si tiene NumPy) y permite elegir un `.exe` concreto, por ejemplo el de un entorno de Anaconda.

2.5 La pantalla de inicio

Sin ningún documento abierto, Pyx muestra la pantalla de bienvenida (figura 2.2). Desde ella creas un documento nuevo (Ctrl + N) o abres uno existente (Ctrl + O). También puedes arrastrar un archivo `.pltx` o `.tex` a la ventana, o hacer doble clic sobre él en el explorador de archivos.

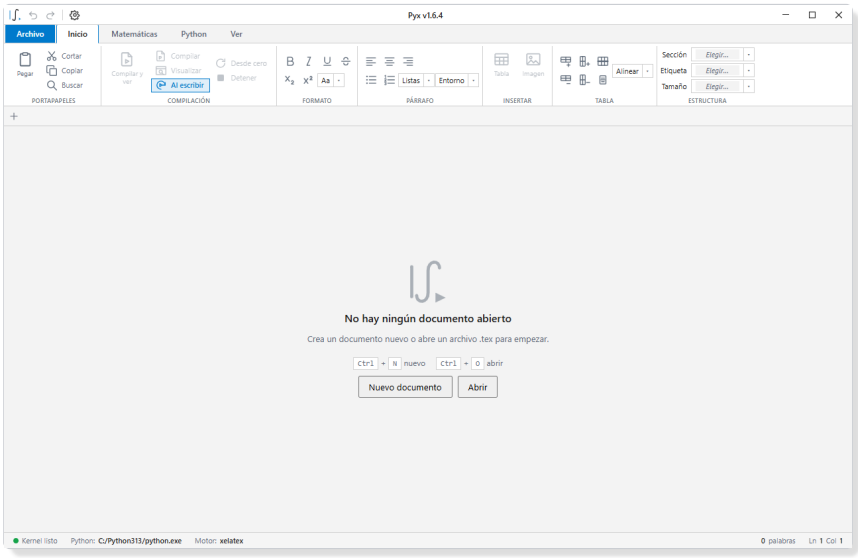


Figura 2.2: La pantalla de inicio, sin documentos abiertos.

3

Recorrido por la interfaz

La figura 3.1 muestra la ventana de Pyx con un informe abierto. Cada número corresponde a una zona que se explica a continuación.

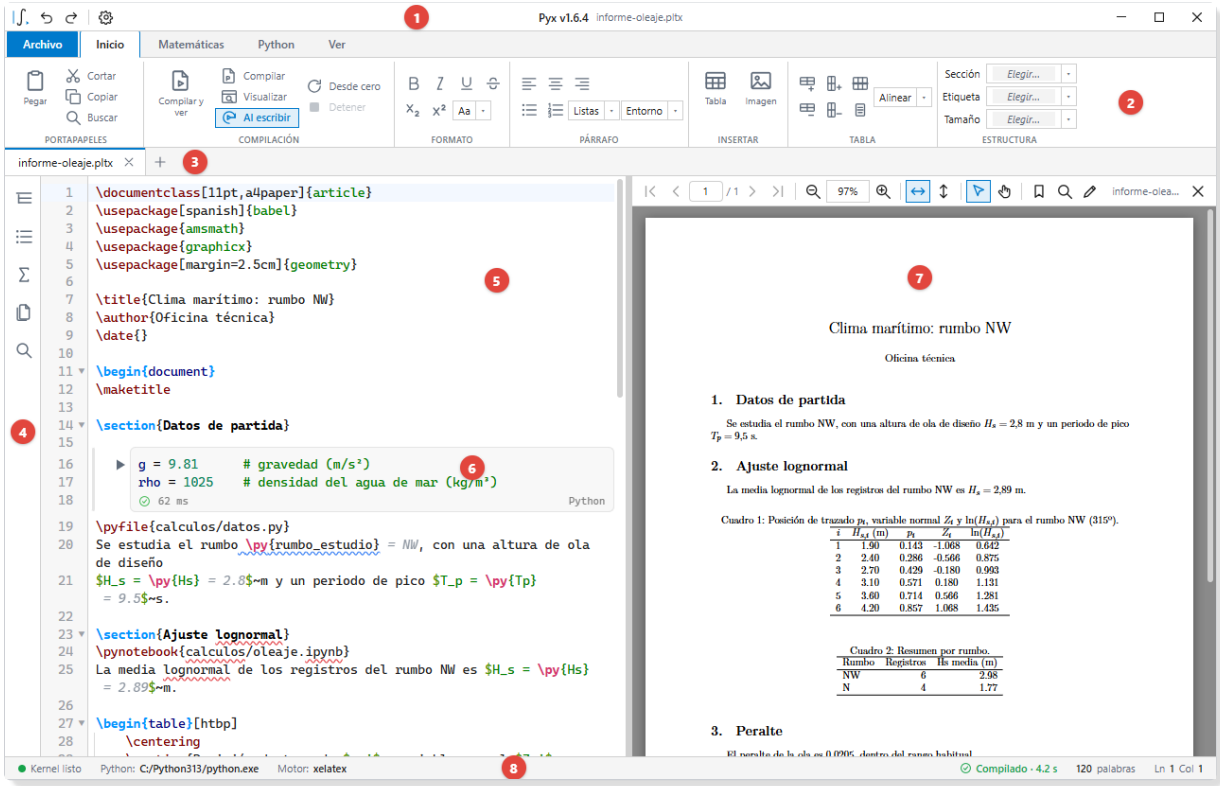


Figura 3.1: La ventana de Pyx: las ocho zonas principales.

1. **Barra de título.** El nombre del documento activo (un punto delante indica cambios sin guardar), los botones de deshacer y rehacer y el engranaje de **Configuración**.
2. **Cinta.** Las herramientas, agrupadas en pestañas: **Archivo**, **Inicio**, **Matemáticas**, **Python** y **Ver**. Se describen en la sección 3.1.
3. **Pestañas de documentos.** Un documento por pestaña; `+` crea uno nuevo. Las imágenes y páginas web que generan las celdas también se pueden abrir como pestañas.
4. **Panel lateral.** Una tira de iconos que abre los paneles de **Estructura**, índice (**TOC**), **Símbolos**, **Archivos** y **Buscar en el proyecto**.
5. **Editor.** El código LaTeX del documento, con números de línea, plegado y resaltado. Junto a cada `\py{}` se ve en gris su valor actual.
6. **Celda de Python.** Un bloque de código con su barra de herramientas y su salida debajo.

7. **Visor de PDF.** El documento compilado, al lado del código. Su barra de herramientas está en la sección 11.

8. **Barra de estado.** El estado del kernel de Python (verde: listo; ámbar: ejecutando; rojo: error), el intérprete y el motor de LaTeX en uso, el resultado de la última compilación, el número de palabras y la posición del cursor. Si el documento importa librerías que el Python en uso no tiene, aparece aquí un aviso **!** Falta: … que permite instalarlas con un clic.

3.1 La cinta

Archivo Nuevo, Abrir, Guardar, Guardar como y Cerrar.

Inicio *Portapapeles* (pegar, cortar, copiar, buscar); *Compilación* (Compilar y ver, Compilar, Visualizar, Al escribir, Desde cero, Detener); *Formato* (negrita, cursiva, subrayado, tachado, subíndice, superíndice, mayúsculas y minúsculas); *Párrafo* (alineación, listas, entornos); *Insertar* (asistentes de Tabla e Imagen); *Tabla* (añadir y quitar filas y columnas, alinear); *Estructura* (secciones, etiquetas y tamaños de letra).

Matemáticas Modo en línea o independiente, subíndices, superíndices, raíces, fracciones, delimitadores que se ajustan, entornos de ecuaciones, funciones, estilos, operadores, acentos y espacios.

Python Nueva celda; Celda actual y Todas las celdas (que pasa a Interrumpir mientras algo se ejecuta); minimizar o expandir todas las celdas; reiniciar el kernel, limpiar y ocultar salidas; las Macros; el Intérprete y el estado del kernel.

Ver Dividir el editor, cerrar paneles, modo zen, vista previa, panel lateral, registro de compilación y terminal.

3.2 El panel lateral

Estructura El archivo actual y los que incluye con `\input`, `\include` o `\subfile`; un clic los abre.

TOC El índice del documento (partes, capítulos, secciones…), con la sección del cursor resaltada, y la lista de tareas pendientes (`% TODO`, `\todo`). Un clic lleva a la línea.

Símbolos Letras griegas, operadores, relaciones, flechas y demás símbolos matemáticos. Un clic los inserta.

Archivos Un explorador de la carpeta del documento: abrir, crear, renombrar y borrar archivos y carpetas.

Buscar en el proyecto Búsqueda y reemplazo en todos los archivos (sección 5.10.1).

Ctrl + Alt + Z activa el **modo zen**: desaparecen la cinta, el panel lateral y la barra de estado, y quedan solo el editor y el PDF. Esc vuelve a la vista normal.

3.3 Temas

Pyx tiene tres temas: **Claro**, **Oscuro** y **Azul**. Se eligen en **Configuración** → **General** → **Tema** y se aplican al momento, también a la ventana auxiliar del visor. El botón de contraste del visor oscurece además el fondo de las páginas del PDF para trabajar de noche.

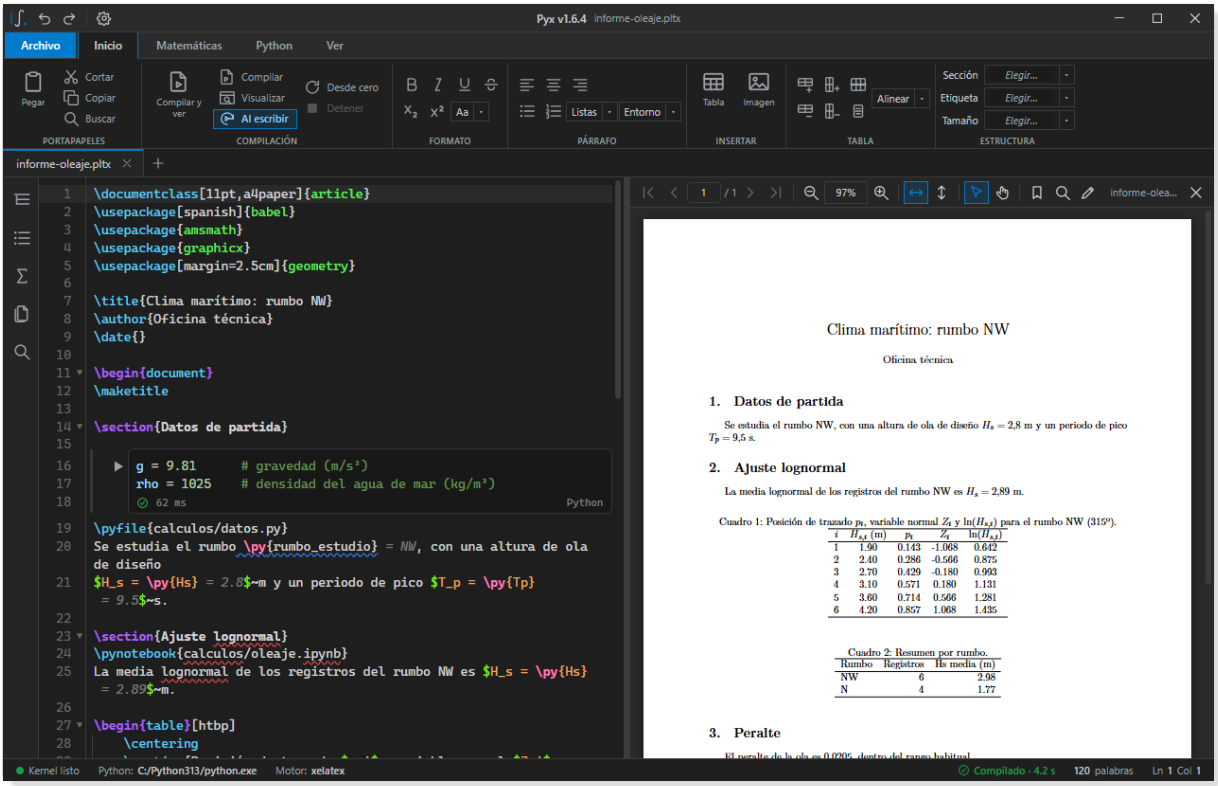


Figura 3.2: El tema oscuro.

4

Documentos y proyectos

4.1 Crear un documento

Nuevo (Ctrl+N) pregunta qué tipo de documento quieres (figura 4.1):

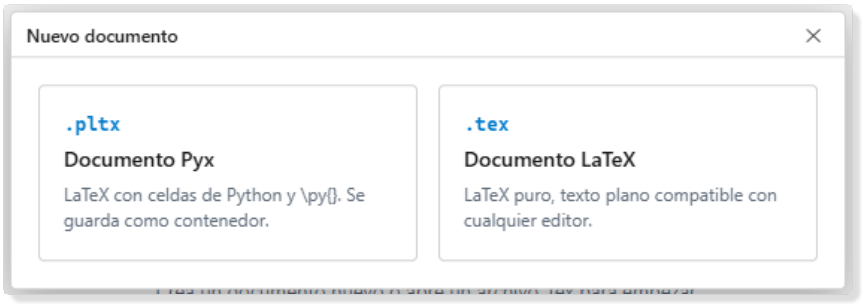


Figura 4.1: Los dos tipos de documento.

Documento Pyx (`.pltx`). LaTeX con celdas de Python y `\py{}`. Es el formato propio de Pyx.

Documento LaTeX (`.tex`). LaTeX puro, en texto plano, compatible con cualquier editor.

4.2 El formato `.pltx`

Un `.pltx` es un contenedor (un archivo ZIP, como los `.docx` de Word) que guarda:

- el código del documento, LaTeX con sus celdas;
- los **resultados de las celdas**: al reabrir el documento se ven los números, las tablas y las figuras sin volver a ejecutar nada, incluso en un equipo sin Python;
- la última compilación: el PDF y sus archivos auxiliares. Al reabrir el documento, el PDF se ve al momento y la siguiente compilación reutiliza las referencias cruzadas.

Para sacar una copia del PDF (para enviarla o imprimirla) usa el botón Guardar una copia del PDF del visor.

> **Nota** Dentro de un `.pltx` las celdas son comentarios de LaTeX (`%#python` y `%#end`), así que el código sigue siendo LaTeX válido. Si añades celdas a un `.tex`, al guardar Pyx te propone guardarlo como `.pltx`.

4.3 Abrir, guardar y cerrar

- Abrir (Ctrl + O) abre `.pltx`, `.tex`, `.bib`, `.sty`, `.py`, `.txt`, imágenes y PDF.
- Guardar (Ctrl + S) guarda el documento activo y, si la compilación automática está activa, compila.
- Guardar como cambia el nombre o la carpeta. Pyx avisa si el documento tiene cambios sin guardar al cerrar su pestaña (Ctrl + W) o la aplicación.
- Si un archivo abierto cambia en el disco (por ejemplo, lo modifica otro programa), Pyx lo detecta.

4.4 Proyectos de varios archivos

Un informe largo se suele dividir en un documento principal y varios capítulos que este incluye con `\input{capitulo1}` o `\include{capitulo1}`. Pyx entiende esa estructura:

- al compilar un capítulo, compila el documento principal que lo incluye;
- las celdas de todos los archivos se ejecutan en el orden en que aparecen en el documento completo;
- las referencias, citas y etiquetas de cualquier archivo abierto se ofrecen al autocompletar;
- el panel **Estructura** muestra los archivos incluidos, y **Buscar en el proyecto** recorre todos.

> **Consejo** Pyx compila en una carpeta de trabajo propia, fuera de la del proyecto. Tu carpeta queda limpia: no se llena de `.aux`, `.log` ni `.synctex.gz`.

5

Escribir LaTeX

5.1 Autocompletado

Al escribir una barra invertida, Pyx propone los comandos que empiezan por lo que llevas escrito, con una breve descripción de cada uno (figura 5.1). Con ↑ ↓ eliges y con `Intro` o `Tab` aceptas. Los comandos con argumentos se insertan con los huecos ya puestos: `\frac{num}{den}` deja seleccionado *num*; `Tab` salta a *den*.

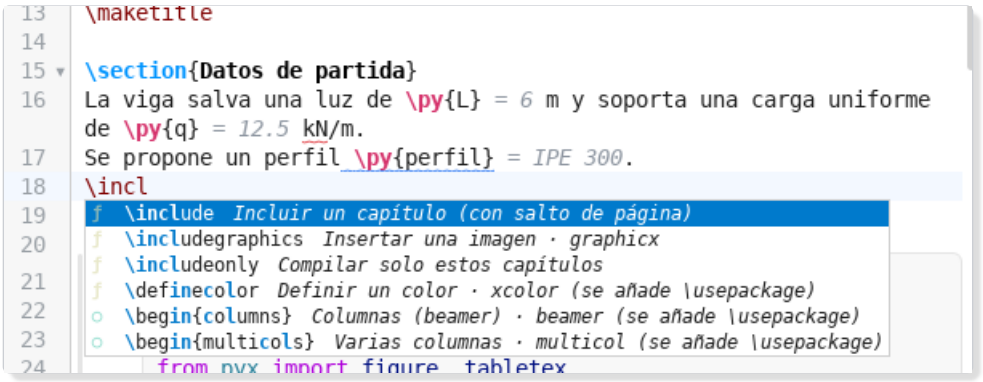


Figura 5.1: Autocompletado de comandos: descripción y paquete de cada uno.

Paquetes automáticos. Si aceptas un comando de un paquete que el documento no carga (por ejemplo `\qty`, de `siunitx`), aparece «(se añade `\usepackage`)» y, al aceptarlo, Pyx añade `\usepackage{siunitx}` al preámbulo.

Tus comandos. Los comandos que define el propio documento con `\newcommand`, `\DeclareMathOperator` o `\def` se ofrecen también, con su número de argumentos.

Entornos. Tras `\begin{` se listan los entornos (figura 5.3). Al elegir uno, Pyx escribe también el `\end` y el contenido habitual: una tabla trae su especificación de columnas; una figura, su `\includegraphics`, su `\caption` y su `\label`. La primera opción, «{ … }», sirve para escribir el nombre de un entorno propio.

Referencias, citas y glosario. Dentro de `\ref{`, `\eqref{`, `\cref{`… se ofrecen las etiquetas de todos los documentos abiertos; dentro de `\cite{`, las claves de los `.bib` que cargas; dentro de `\gls{`, las entradas del glosario y las siglas.

Valores de Python. Al escribir `\py{` se abre la lista de las variables que hay (figura 5.2): las que definen las celdas del documento y las de los cuadernos y archivos `.py` que vincula, con el archivo del que viene cada una. Funciona en cualquier punto de la expresión (`\py{filas(rumbo_` también completa) y, tras un punto, ofrece los atributos del valor.

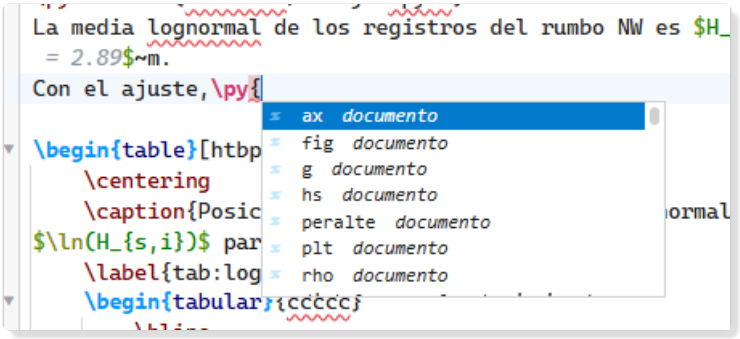


Figura 5.2: Dentro de `\py{`: las variables del documento, del cuaderno y del archivo `.py`.

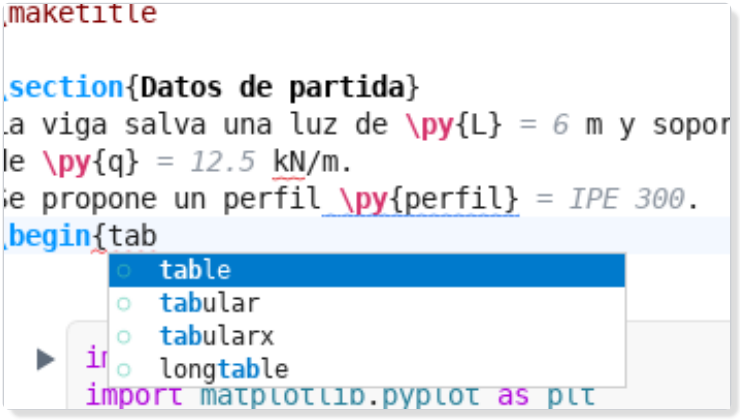


Figura 5.3: Al escribir `\begin{tab` se listan los entornos de tabla.

Consejo El autocompletado se ajusta en **Configuración** → **Autocompletado**: si aparece solo al escribir o únicamente con `Ctrl`+`Espacio`, si distingue mayúsculas y si muestra iconos.

5.2 La cinta de formato

La pestaña **Inicio** aplica formato a la selección (o deja el cursor dentro, si no hay selección): **negrita** (`Ctrl`+`B`), *cursiva* (`Ctrl`+`I`), subrayado, tachado, subíndice, superíndice y cambio de mayúsculas. El grupo *Estructura* inserta secciones (con su nivel indicado: Pt, Cap, S1, S2…), etiquetas, referencias y tamaños de letra. La pestaña **Matemáticas** hace lo mismo con las fórmulas.

5.3 Tablas

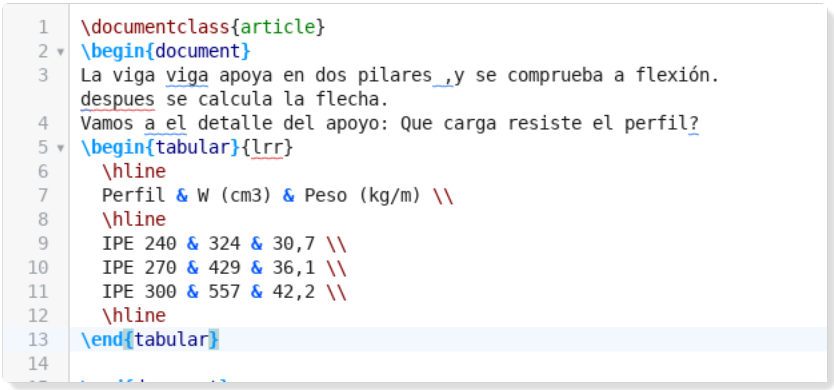
Asistente. Tabla abre un asistente: número de filas y columnas, alineación de cada columna, encabezado, líneas de `booktabs` o `\hline`, pie y etiqueta. Genera el LaTeX limpio.

Editar una tabla. Con el cursor dentro de un `tabular`, el grupo *Tabla* de la cinta añade o elimina filas y columnas (manteniendo la especificación de columnas), pone o quita `\hline` y cambia la alineación de la columna.

Pegar desde Excel. Copia unas celdas de Excel, LibreOffice Calc, Word o una página web y pégalas en el documento con `Ctrl`+`V`: Pyx las convierte en un `tabular` (figura 5.4). Los números

se alinean a la derecha, el texto a la izquierda, los caracteres especiales (`&`, `%`, `_`…) se escapan y, si el documento carga `booktabs`, se usan sus reglas. Si preferías el texto tal cual, Ctrl+Z lo deja como se copió. Solo se convierte lo que de verdad es una tabla: el código con sangría de tabuladores, o el que se copia de GitHub o de una página de documentación, se pega como texto.

Lo que no viaja al pegar Al pegar, Pyx quita los caracteres invisibles que traen Word, Teams, los PDF y muchas webs (espacios de anchura cero, guiones blandos, marcas de dirección), que rompen un comando o una palabra sin que se vean, y separa las ligaduras de los PDF (*fi* pasa a *fi*). Dentro de una celda de Python, además, los espacios de no separación se vuelven espacios normales: Python no los admite en el código. Y si pegas una celda copiada de otro documento o del mismo, recibe un identificador propio, para que no comparta la salida con la original.



```
\documentclass{article}
\begin{document}
La viga viga apoya en dos pilares ,y se comprueba a flexión.
despues se calcula la flecha.
Vamos a el detalle del apoyo: Que carga resiste el perfil?
\begin{tabular}{lrr}
  \hline
  Perfil & W (cm3) & Peso (kg/m) \\
  \hline
  IPE 240 & 324 & 30,7 \\
  IPE 270 & 429 & 36,1 \\
  IPE 300 & 557 & 42,2 \\
  \hline
\end{tabular}

```

Figura 5.4: Tres filas pegadas desde una hoja de cálculo se convierten en un `tabular`.

5.4 Imágenes y símbolos

Imagen abre un asistente para elegir la imagen, su ancho, el pie y la etiqueta, y genera la figura. El panel **Símbolos** inserta letras griegas, operadores, flechas y demás símbolos con un clic, como en TeXstudio.

5.5 Fórmulas: vista previa al pasar el ratón

Al dejar el ratón sobre una fórmula (`$…$`, `\[ … \]`, `equation`, `align`…) aparece dibujada, sin compilar (figura 5.5). Si la fórmula contiene un `\py{}`, se muestra con su valor actual.

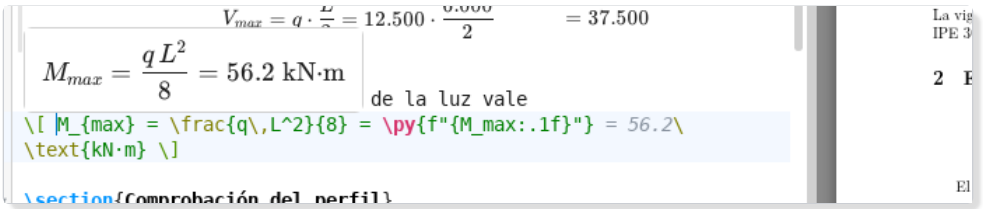


Figura 5.5: Vista previa de una fórmula con el valor calculado por Python.

5.6 Navegar por el documento

Ctrl+clic en una referencia. Sobre `\ref{fig:viga}`, lleva al `\label{fig:viga}`, aunque esté en otro documento abierto.

Alt+← y Alt+→. Vuelven al sitio donde estabas antes de un salto (un clic lejano, una referencia, un resultado de búsqueda, un salto desde el PDF) y avanzan de nuevo. Funciona también entre documentos.

Marcadores. Ctrl+F2 pone o quita un marcador en la línea; F2 y Mayús+F2 saltan al siguiente y al anterior.

Plegado. Las flechas del margen pliegan entornos y secciones.

Índice. El panel **TOC** lleva a cualquier sección con un clic.

5.7 Edición inteligente

- Al renombrar `\begin{itemize}`, su `\end{itemize}` cambia a la vez; Ctrl+Z deshace las dos cosas de una vez.
- Intro dentro de una lista añade un `\item` nuevo; detrás de un `\begin{…}` abre el entorno.
- Ctrl+T comenta o descomenta las líneas seleccionadas.
- Alt+↑ y Alt+↓ mueven la línea (o la celda entera) arriba o abajo.
- Los paréntesis, corchetes y llaves se cierran solos y se resalta su pareja.
- Ctrl+Alt+D divide el editor en dos paneles con el mismo documento o con documentos distintos.

5.8 Comprobaciones mientras escribes

Pyx subraya los errores de estructura antes de compilar, como el servidor de lenguaje *texlab*: llaves sin cerrar, `\begin` y `\end` que no coinciden, etiquetas repetidas, `\ref` a etiquetas que no existen y `$` sin cerrar. Tras compilar, los errores y avisos del registro también se marcan en su línea. F8 y Mayús+F8 recorren todos los problemas. Si tienes ChkTeX instalado, puedes añadir sus avisos de estilo en **Configuración** → **Compilación**.

5.9 Ortografía y gramática

El corrector ortográfico subraya en rojo las palabras que no reconoce, en español o en inglés (el idioma se elige en **Configuración** → **Comprobación del lenguaje**). Ignora los comandos, las fórmulas, el código y los comentarios, e incluye el vocabulario técnico de la ingeniería. Clic derecho sobre una palabra para ver sugerencias o añadirla al diccionario.

La revisión gramatical subraya en azul (figura 5.6); al dejar el ratón encima se ve el motivo:

- palabras repetidas («la viga viga»);

- espacios que sobran antes de la puntuación o faltan después («pilares ,y»);
- minúscula después de un punto (sin confundir abreviaturas como «p. ej.» o «fig.»);
- en español, «a el» y «de el» (se escribe «al» y «del»), y los signos de apertura ¿ ¡ que faltan.

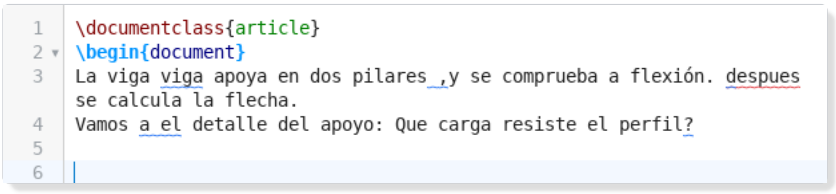


Figura 5.6: Ortografía (rojo) y gramática (azul).

LanguageTool

Para una revisión gramatical completa, Pyx puede usar *LanguageTool*, el motor de LibreOffice. En **Configuración** → **Comprobación del lenguaje** → **Servidor de LanguageTool** escribe:

- `http://localhost:8081` si tienes tu propio servidor (`java -jar languagetool-server.jar --port 8081`); el texto no sale de tu equipo;
- `https://api.languagetool.org` para el servidor público, que recibe el texto del documento.

Sus avisos aparecen como problemas (`F8`), cada uno con las correcciones propuestas, que se aplican con un clic.

5.10 Buscar y reemplazar

`Ctrl`+`F` abre la búsqueda en el documento (o en el PDF, si el foco está en el visor), con mayúsculas, palabra completa y expresiones regulares; la flecha de la izquierda despliega el reemplazo.

5.10.1 Buscar en todo el proyecto

`Ctrl`+`Mayús`+`F` abre el panel **Buscar en el proyecto** (figura 5.7). Busca en todos los archivos de la carpeta del proyecto (`.tex`, `.pltx`, `.bib`, `.py`, `.sty`…) y agrupa los resultados por archivo; un clic abre el archivo con la coincidencia seleccionada. Los documentos abiertos se buscan tal como están en el editor, con los cambios sin guardar.

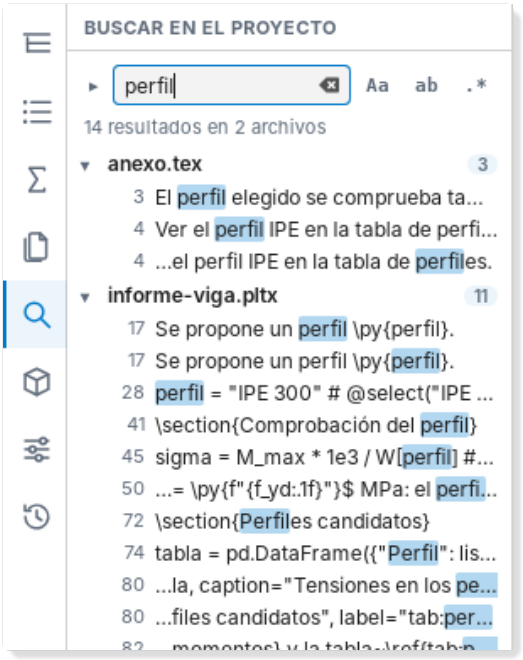


Figura 5.7: Resultados agrupados por archivo.

La flecha despliega el reemplazo. Todo reemplaza en todos los archivos: en los abiertos, el cambio se hace en el editor (se puede deshacer); los demás archivos de texto se guardan directamente, tras confirmarlo.

6

Celdas de Python

Una celda es un bloque de Python dentro del documento. En el archivo se escribe entre dos comentarios de LaTeX:

```
%#python
L = 6.0
q = 12.5
M = q * L**2 / 8
%#end
```

En el editor no se ven esas dos líneas: Pyx dibuja la celda como en Jupyter o VS Code, con su barra de herramientas arriba y la salida debajo. En el PDF la celda no aparece; aparece lo que tú decidas mostrar (capítulo 7).

6.1 Crear y ejecutar celdas

Nueva celda: `Ctrl`+`Alt`+`C` o Nueva celda. Se crea vacía, con el cursor dentro.

Ejecutar la celda: `Ctrl`+`Intro`, o el botón a su izquierda.

Ejecutar y avanzar: `Mayús`+`Intro` ejecuta la celda y lleva el cursor debajo.

Ejecutar todas: `Ctrl`+`Alt`+`Intro` reinicia el espacio de nombres y ejecuta todas las celdas desde el principio.

Interrumpir: el botón que sustituye al mientras la celda se ejecuta, o Interrumpir en la cinta. Las variables calculadas hasta ese momento se conservan. Si la celda no responde, un segundo clic termina el proceso.

La barra de la celda (visible al poner el cursor en ella) tiene, como en VS Code: ejecutar las celdas anteriores, ejecutar esta y las siguientes, dividir la celda por el cursor, más acciones (cortar, copiar, insertar arriba o abajo, borrar la salida, copiar la salida) y eliminar.

> **Consejo** Un clic en la barra azul de la izquierda de una celda la pliega o la despliega; lo mismo con la de su salida. Minimizar todas en la pestaña **Python** pliega todas a la vez.

6.2 Las salidas

Debajo de cada celda aparece lo que produce, en el orden en que lo produce:

- el texto de `print()` y los avisos;

- el valor de la última línea, si es una expresión (un número, una lista, un DataFrame…);
- las tablas de pandas, con formato (figura 6.1);
- las gráficas de Matplotlib, Plotly, Bokeh o Altair (figura 6.2);
- fórmulas en LaTeX (SymPy, handcalcs), HTML, Markdown, imágenes, audio y vídeo.

El número entre corchetes es el contador de ejecución, como en Jupyter, y junto al resultado se indica el tiempo que tardó la celda. Las imágenes y páginas HTML se pueden abrir en una pestaña o en el navegador.



Figura 6.1: Un DataFrame de pandas.

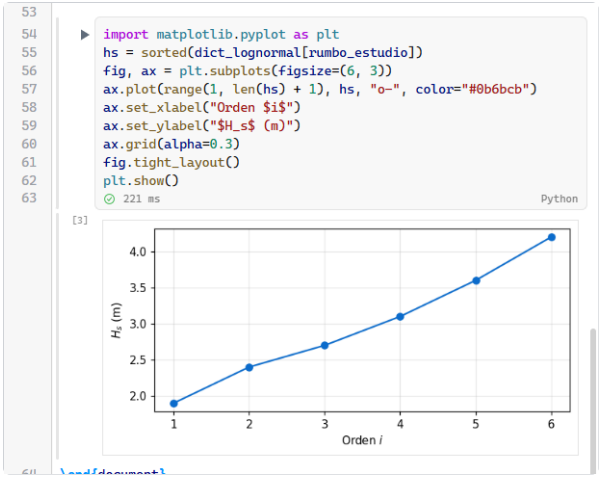


Figura 6.2: Una gráfica de Matplotlib.

> **Nota** Las salidas se guardan dentro del `.pltx`: al reabrir el documento se ven sin ejecutar nada. Si la celda cambia, su resultado se marca como «desactualizado» para que nunca pase por actual un número que ya no corresponde a su fórmula.

6.3 Errores

Si una celda falla, la salida muestra el error con su traza, en color y con los números de línea del documento: un clic en «línea 12» lleva a esa línea. El error aparece también en el panel **Problemas** del registro.

Instalar un paquete que falta

Si el error es `ModuleNotFoundError` (la celda importa una librería que no está instalada), aparece el botón Instalar … (figura 6.3). Pyx instala el paquete en el Python que usa, reinicia el kernel y basta con volver a ejecutar la celda. Además, al abrir un documento, Pyx comprueba qué librerías importan sus celdas y, si falta alguna, lo avisa en la barra de estado.

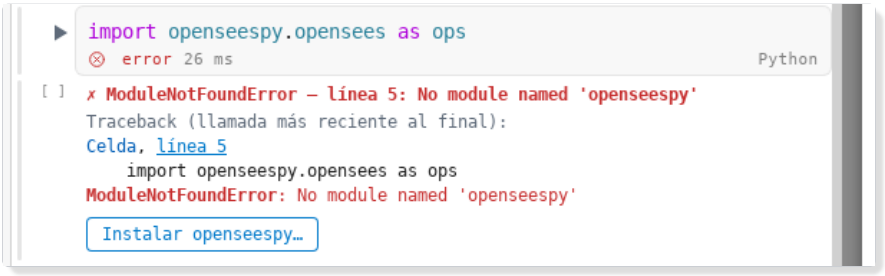


Figura 6.3: Una librería que falta se instala con un clic.

6.4 Autocompletado y ayuda de Python

Dentro de una celda, el autocompletado lo resuelve *jedi* (el mismo motor que usan IPython y JupyterLab) sobre las variables reales del kernel: `np.lin` ofrece `linspace` y `linalg` (figura 6.4); `df.` ofrece los métodos del DataFrame que has creado; dentro de una cadena, como en `pd.read_csv("da`, ofrece los archivos de la carpeta del proyecto.

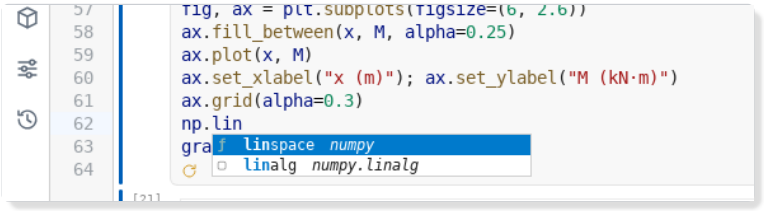


Figura 6.4: Autocompletado con jedi dentro de una celda.

Al escribir el paréntesis de una llamada aparece la **firma** de la función, con el parámetro que estás escribiendo en negrita, y su documentación (figura 6.5). Al dejar el ratón sobre un nombre, se muestra su ayuda. `Esc` la cierra.

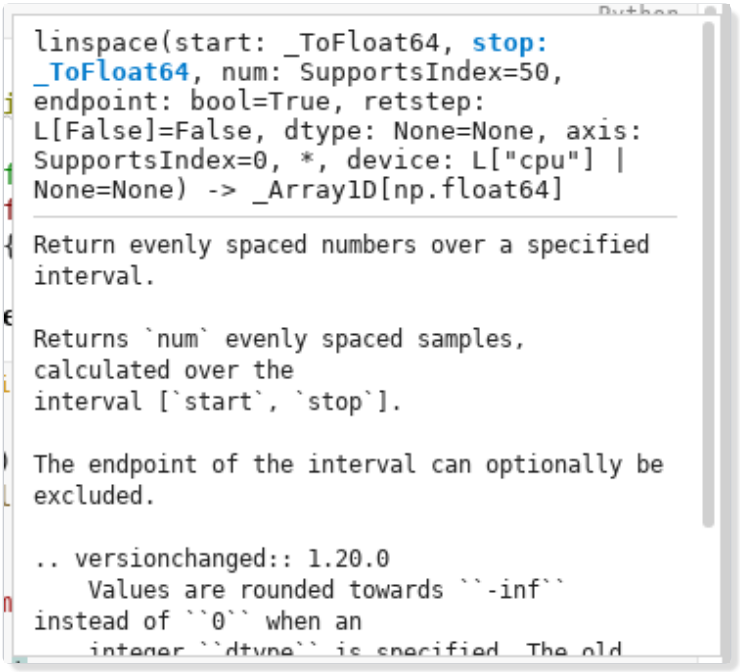


Figura 6.5: La firma de `np.linspace` al escribir el primer argumento.

6.5 El kernel

Las celdas se ejecutan en un intérprete de Python persistente, el *kernel*: las variables y los `import` se conservan entre ejecuciones, como en Jupyter. Las celdas se ejecutan desde la carpeta del documento principal, así que `open("datos.csv")` lee el archivo que está junto al documento.

- Reiniciar (`Ctrl`+`Alt`+`R`) empieza con un espacio de nombres vacío.
- Limpiar salidas borra las salidas de todas las celdas; Ocultar salidas las oculta sin borrarlas (los cálculos siguen yendo al PDF).
- Si una celda modifica una librería (por ejemplo `math.pi = 3`), Pyx lo detecta y, al recompilar desde cero, sustituye el proceso por uno nuevo para que el cambio no contamine el siguiente cálculo.

Pyx admite las magias de handcalcs (`%%render`, `%%tex`), `display()`, los objetos con `_repr_html_`, `_repr_latex_` y similares, `asyncio` y las celdas con `await`.

7

Del cálculo al documento

Las celdas calculan; este capítulo explica cómo llevar sus resultados al PDF. Hay cuatro caminos, de menor a mayor elaboración: un valor en el texto, un texto condicional, una tabla o figura completa, y el desarrollo del cálculo con handcalcs.

7.1 Un valor en el texto: `\py{}`

`\py{expresión}` escribe en el PDF el resultado de cualquier expresión de Python, evaluada con las variables de las celdas:

```
La viga salva una luz de \py{L} m y el momento máximo es
de \py{round(M, 1)} kN·m.
```

Para dar formato a los números se usan las *f-strings* de Python: `\py{f"{M:.2f}"}` escribe dos decimales; `\py{f"{N:,.0f}"}`, separadores de miles; `\py{f"{x:.3e}"}`, notación científica.

Lo que escribe depende de lo que devuelve la expresión:

Un número con todas sus cifras significativas, nunca en notación `1e+05`; un `NaN` o un infinito dan un error en lugar de colarse en el PDF.

Un texto tal cual: es tu propio LaTeX. Por eso una función que devuelve las filas de una tabla (`"1 & 2.5 \\..."`) se coloca dentro de un `tabular`.

Una tabla de pandas (un `DataFrame` o una `Series`) se escribe como un `tabular` completo, como la que se ve bajo una celda: `\py{resumen}` dentro de un entorno `table`. Para más de 500 filas, muestra una parte (`\py{df.head(20)}`) o usa `tabletex`.

Una cantidad de Pint, una expresión de SymPy o cualquier objeto que sepa dibujarse en LaTeX se compone como fórmula.

> **Si una variable recibe dos valores** Cuando dos celdas, o dos archivos vinculados (capítulo 8), dan valor a la misma variable, cada `\py{}` muestra el valor que le dio la celda o el archivo *más próximo por encima*, como al leer un cuaderno de arriba abajo. Un `\py{}` situado antes de todos ellos (un resumen al principio del informe) muestra el valor final.

En el editor, a la derecha de cada `\py{}` se ve en gris su valor actual, sin compilar (al estilo de Mathcad). Esos valores se actualizan al ejecutar celdas y al compilar; se pueden ocultar en **Configuración** → **Celdas Python**.

> **Nota** Mientras escribes, los valores en gris de las expresiones que modificarían datos (por ejemplo `lista.pop()` o `next(iterador)`) no se calculan: dicen «se calcula al compilar». Así, mirar el documento nunca cambia los resultados.

7.2 Un texto u otro: `\pyif`

`\pyif{condición}{si se cumple}{si no}` elige el texto según una condición de Python. El tercer argumento es opcional:

```
El perfil \pyif{sigma <= f_yd}{\textbf{cumple}}{\textbf{no cumple}}
con un aprovechamiento del \py{f"{100*sigma/f_yd:.0f}"} \%.
```

Los dos textos pueden contener cualquier LaTeX, incluso otros `\py{}` y `\pyif`.

7.3 Tablas y figuras completas: el módulo `pyx`

Las celdas pueden importar las ayudas de Pyx con `from pyx import …`:

`figure(nombre)` guarda la figura actual de Matplotlib en la carpeta de imágenes del proyecto (`xref/`) y devuelve su ruta, para usarla en `\includegraphics{\py{ruta}}`.

`figtex(nombre, caption=…, label=…, width=0.8)` devuelve el entorno `figure` completo: se coloca con un solo `\py{}`.

`tex(objeto)` convierte en LaTeX una expresión de SymPy, una cantidad de Pint con sus unidades, una matriz de NumPy o un DataFrame de pandas.

`tabletex(df, caption=…, label=…, prec=…)` devuelve una tabla completa a partir de un DataFrame.

`texesc(texto)` escapa los caracteres especiales de LaTeX de un texto cualquiera.

7.4 Un ejemplo completo

Lo que sigue lo ha calculado una celda de este mismo manual al compilarlo. La celda es:

```
P = 15.0          # carga (kN)
L = 3.0           # vuelo (m)
E = 210e3         # módulo de elasticidad (MPa)
I = 8356e4        # inercia del IPE 300 (mm4)

flecha = P * 1e3 * (L * 1e3)**3 / (3 * E * I)      # mm
admisible = L * 1e3 / 300                           # mm (L/300)
# ... la gráfica con Matplotlib ...
ruta_flecha = figure("flecha")
```

Y el texto del documento es:

```
La flecha en el extremo es \py{f"{flecha:.2f}"} mm, \pyif{flecha <= admisible}
{menor}{mayor} que la admisible de \py{f"{admisible:.1f}"} mm.
```

que en el PDF se convierte en:

> La flecha en el extremo es 7.69 mm, menor que la admisible de 10.0 mm. La figura 7.1 muestra la deformada.

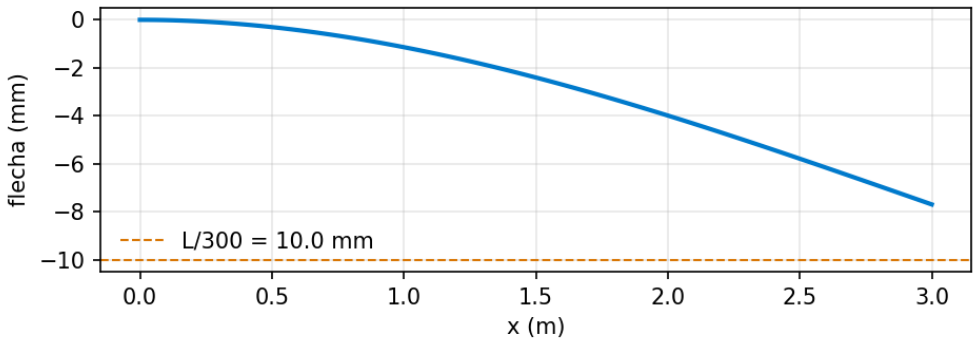


Figura 7.1: Deformada de la ménsula, dibujada por una celda de Python al compilar este manual.

Si mañana cambia la carga a 20 kN, basta con editar la celda y compilar: el número, la comparación y la gráfica cambian a la vez.

7.5 El desarrollo del cálculo: handcalcs

Para que el PDF muestre el cálculo paso a paso, como se haría a mano, usa la biblioteca *handcalcs*: importa `handcalcs.render` en una celda anterior y empieza la celda con `%%render`:

```
%%render
M_max = q * L**2 / 8
V_max = q * L / 2
```

handcalcs escribe cada fórmula con sus valores sustituidos y su resultado, y Pyx la coloca en el PDF en el sitio de la celda. `%%tex` hace lo mismo y, además, imprime el código LaTeX generado. Estas son las únicas celdas que ponen contenido en el PDF por sí mismas; todas las demás solo aparecen a través de `\py{}` y `\pyif`.

> **Atención** Como en Jupyter, `%%render` solo funciona después de `import handcalcs.render`. Si falta, la celda falla con un mensaje que lo explica.

8

Cuadernos de Jupyter y archivos de Python

Muchos cálculos ya existen fuera del documento: en un cuaderno de Jupyter que trabajas en VS Code, o en un archivo `.py`. Pyx los usa tal como están, sin copiar código al informe:

```
\pyfile{calculos/datos.py}
\pynotebook{calculos/oleaje.ipynb}
La altura de ola de diseño es $H_s = \py{Hs}$~m.
```

`\pynotebook{…}` y `\pyfile{…}` son el mismo comando y admiten los dos tipos de archivo; si no se escribe extensión, se entiende `.ipynb`. La ruta es relativa a la carpeta del documento principal, como en `\input`. En el PDF el comando no escribe nada: sirve para que, desde ese punto, `\py{}`, `\pyif` y las celdas del documento usen las variables del archivo como si fueran suyas, con la misma sintaxis de siempre. `Ctrl`+clic sobre la ruta abre el archivo.

8.1 Qué se puede usar

Todo lo que el cuaderno o el archivo deja definido: números y textos, listas y diccionarios, tablas de pandas (que `\py{}` escribe como tablas de LaTeX), arrays de NumPy y **funciones**. Una función del cuaderno se puede llamar desde el documento, y se lleva con ella todo lo que usa por dentro (otras funciones, diccionarios, módulos como `np`), aunque el documento no lo nombre:

```
\begin{tabular}{ccccc}
  \hline
  $i$ & $H_{s,i}$ (m) & $p_i$ & $Z_i$ & $\ln(H_{s,i})$ \\
  \hline
  \py{filas_ln(rumbo_estudio, len(dict_lognormal[rumbo_estudio]))}
  \hline
\end{tabular}
```

Las imágenes del cuaderno no se traen: una figura se guarda como archivo en el cuaderno y se coloca con `\includegraphics`, como siempre.

8.2 Sin volver a ejecutarlos

Un cuaderno puede tardar minutos en ejecutarse. Pyx no lo repite: toma los valores que ya tiene, por este orden.

1. **De Jupyter, en vivo.** Si el cuaderno está abierto en VS Code, JupyterLab o Jupyter Notebook y ya lo has ejecutado, Pyx copia de ese kernel las variables que usa el documento, en unas décimas de segundo. En el cuaderno no se ejecuta nada ni queda rastro: ni en su historial ni

entre sus variables. Si desde la última compilación no has ejecutado nada en Jupyter, ni siquiera se copian de nuevo.

2. **De los valores guardados.** Con Jupyter cerrado, Pyx usa los últimos valores que tomó, mientras el código del archivo no cambie. También sirven para un `\py{}` nuevo que lea otra variable pequeña del archivo.
3. **Ejecutándolo una vez.** Solo si no hay ninguna de las dos cosas (la primera vez con Jupyter cerrado, o porque has cambiado el código), Pyx ejecuta el archivo, desde su propia carpeta, como Jupyter. Sus valores quedan guardados y la siguiente compilación ya no lo ejecuta.

Desde cero y Ejecutar todo tampoco vuelven a ejecutar los archivos vinculados. El registro de la compilación dice, para cada uno, de dónde salieron sus valores y cuánto se tardó (figura 10.1), y los avisos señalan las variables que Jupyter todavía no tiene (las de una celda que no has ejecutado allí).

8.3 Archivos `.py` y sus celdas

Un archivo `.py` se ejecuta entero, o por bloques si usa los separadores `# %%` de VS Code, Spyder y PyCharm (también `# In[ ]:`). Con bloques, Pyx vuelve a ejecutar solo los que cambian, como con las celdas del documento; los bloques `# %% [markdown]` son texto y se ignoran.

8.4 Varios archivos y variables con el mismo nombre

Se pueden vincular tantos archivos como se quiera, cada uno en su sitio del documento. Si dos dan valor a la misma variable, cada `\py{}` muestra la del archivo *más próximo por encima*:

```
\pyfile{datos.py}                 % Hs = 2.8
La ola de diseño es \py{Hs} m.        % 2.8
\pynotebook{oleaje.ipynb}         % Hs = 2.89 (la media lognormal)
La media de los registros es \py{Hs} m.   % 2.89
```

Las celdas del documento siguen la misma regla: una celda situada después de `\pynotebook{oleaje.ipynb}` lee el `Hs` del cuaderno. El orden en que se usan los archivos es el orden en que aparecen en el documento, que puede ser distinto del orden en que los creaste.

9

Ejecución reactiva

Pyx sabe qué variables define y qué variables usa cada celda. Con esa información construye un *grafo de dependencias*, la misma idea que usa marimo: si cambias una celda, solo hay que volver a ejecutar esa celda y las que dependen de ella, no todo el documento.

9.1 Qué se ejecuta

Imagina un informe con cinco celdas: los datos, la carga de un archivo grande, una simulación que tarda medio minuto, una gráfica y una tabla resumen.

- Si cambias el **texto** del informe, no se ejecuta ninguna celda.
- Si cambias la **gráfica**, solo se ejecuta la gráfica: la simulación no se repite.
- Si cambias los **datos**, se ejecutan las celdas que usan esos datos, directa o indirectamente.

Además, si una celda vuelve a dar exactamente el mismo resultado (por ejemplo, vuelves a escribir `g = 9.81`), las que dependen de ella no se ejecutan: Pyx compara los valores y corta la cadena (*early cutoff*).

El registro de cada compilación indica cuántas celdas se han ejecutado y cuántas había, por ejemplo «Celdas Python : 220 ms (1 ejecutada de 5)».

9.2 Celdas desactualizadas

Cuando editas una celda sin ejecutarla, las celdas que dependen de ella se marcan en cascada con «desactualizada: depende de una celda modificada» (figura 9.1). La celda editada indica «resultado desactualizado». Así ves de un vistazo qué resultados ya no corresponden al código.



Figura 9.1: Tras cambiar la luz en la primera celda, las celdas que la usan se marcan como desactualizadas.

9.3 Mayúsculas+Intro, reactivo

Con la ejecución reactiva activada (es lo habitual), `Mayús`+`Intro` ejecuta la celda y, después, las celdas que dependen de lo que ha cambiado, y solo esas. Si la celda necesita una variable de otra celda que todavía no se ha ejecutado, esa celda se ejecuta primero.

Si prefieres el comportamiento de Jupyter (ejecutar solo la celda pulsada), desactiva **Configuración** → **Celdas Python** → **Ejecución reactiva**. Las celdas que dependen de ella se marcarán entonces como desactualizadas, y la siguiente compilación las pondrá al día. La compilación siempre ejecuta únicamente lo necesario, esté activada o no esta opción.

9.4 Garantías

Ejecutar solo una parte del documento tiene que dar los mismos números que ejecutarlo entero. Pyx lo garantiza así:

- Modificar un objeto cuenta como definirlo: si una celda hace `lista.append(x)`, al editarla se vuelve a crear antes la lista, para que no crezca dos veces.
- Las variables que deja de definir una celda editada o borrada se eliminan, y las celdas que las usaban se vuelven a ejecutar (y fallan, como fallarían en una ejecución completa).
- Los números aleatorios se tratan como un estado común: si una celda usa `np.random`, se repite también la que fijó la semilla.
- Si una celda lee un archivo de datos (un Excel, un CSV) y el archivo cambia, la celda se vuelve a ejecutar aunque su código sea el mismo.

- Lo que el análisis no puede seguir con seguridad (`exec`, `global`, las magias de IPython, `from módulo import *`) hace que Pyx vuelva a la regla clásica: ejecutar desde la primera celda que cambió o, si hace falta, todo.

Nota Esta parte de Pyx se comprueba automáticamente con cientos de documentos generados al azar: para cada uno, ejecutar solo lo que dice el grafo da el mismo resultado, variable por variable, que ejecutar el documento entero.

Consejo Recompilar desde cero (`Ctrl` + `Alt` + `Mayús` + `B`) siempre empieza con un espacio de nombres vacío y ejecuta todas las celdas. Es la referencia si alguna vez dudas de un resultado.

10

Compilar

Compilar es convertir el documento en PDF. Pyx hace en una sola acción todo lo que hace falta: guarda, ejecuta las celdas necesarias, sustituye los `\py{}` y `\pyif`, ejecuta LaTeX las veces necesarias y muestra el PDF.

10.1 Las formas de compilar

Compilar y ver (Ctrl + Mayús + B) compila y abre el visor si estaba cerrado.

Compilar compila sin cambiar la vista.

Al escribir compila sola cada vez que dejas de escribir un momento. Pyx espera al menos el triple de lo que tardó la última compilación, para que un documento muy pesado no se esté recompilando continuamente. El retardo se ajusta en **Configuración** → **Compilación**.

Desde cero (Ctrl + Alt + Mayús + B) empieza con un kernel nuevo y sin los archivos auxiliares de compilaciones anteriores.

Detener interrumpe la compilación en curso; se conserva el PDF anterior.

Mientras compila, la barra de estado indica la fase: «Preparando…», «Ejecutando Python…», «Componiendo LaTeX…» o «Cargando PDF…». Al terminar indica el resultado y el tiempo total.

10.2 Rápido: el motor precalentado

Al terminar cada compilación, Pyx deja un proceso de LaTeX con el preámbulo (paquetes y fuentes) ya cargado, esperando. Si la siguiente compilación tiene el mismo preámbulo, solo se compone el cuerpo del documento: en un informe típico, cada compilación pasa de unos 0,7 s a unos 0,1 s. Si cambias el preámbulo, un `.sty` o el motor, Pyx compila de la forma normal. Se puede desactivar en **Configuración** → **Compilación** → **Velocidad**.

10.3 Motores

Pyx funciona con **xelatex** (el predeterminado), **pdflatex**, **lualatex** y **Tectonic**. El motor se elige por documento en **Configuración** → **Compilación** → **Motor**. Con Tectonic no hace falta ninguna distribución de LaTeX: descarga los paquetes que necesita cada documento la primera vez que lo compila.

10.4 Bibliografía, índices y glosarios

Pyx ejecuta solo, entre las pasadas de LaTeX, las herramientas auxiliares que el documento necesita, y solo cuando cambia lo que leen (las mismas reglas que *latexmk*):

- **BibTeX** o **Biber** (con `biblatex`) para las citas;
- **makeindex** para el índice alfabético (`makeidx`, `imakeidx`);
- las listas del paquete **glossaries** (glosario, siglas, símbolos), ordenadas como lo haría `makeglossaries`, sin necesidad de Perl.

También repite LaTeX cuando las referencias cruzadas o el índice cambian, hasta que se estabilizan.

10.5 El registro y los problemas

El botón Registro de la pestaña **Ver** abre, bajo el editor, el panel de compilación con dos pestañas (figura 10.1):

Problemas los errores y avisos, de LaTeX y de las celdas, con su archivo y su línea; un clic lleva a la línea.

Salida el registro completo de LaTeX y, al final, los tiempos de Pyx: cuánto tardaron las celdas (y cuántas se ejecutaron) y cuánto el motor de LaTeX.

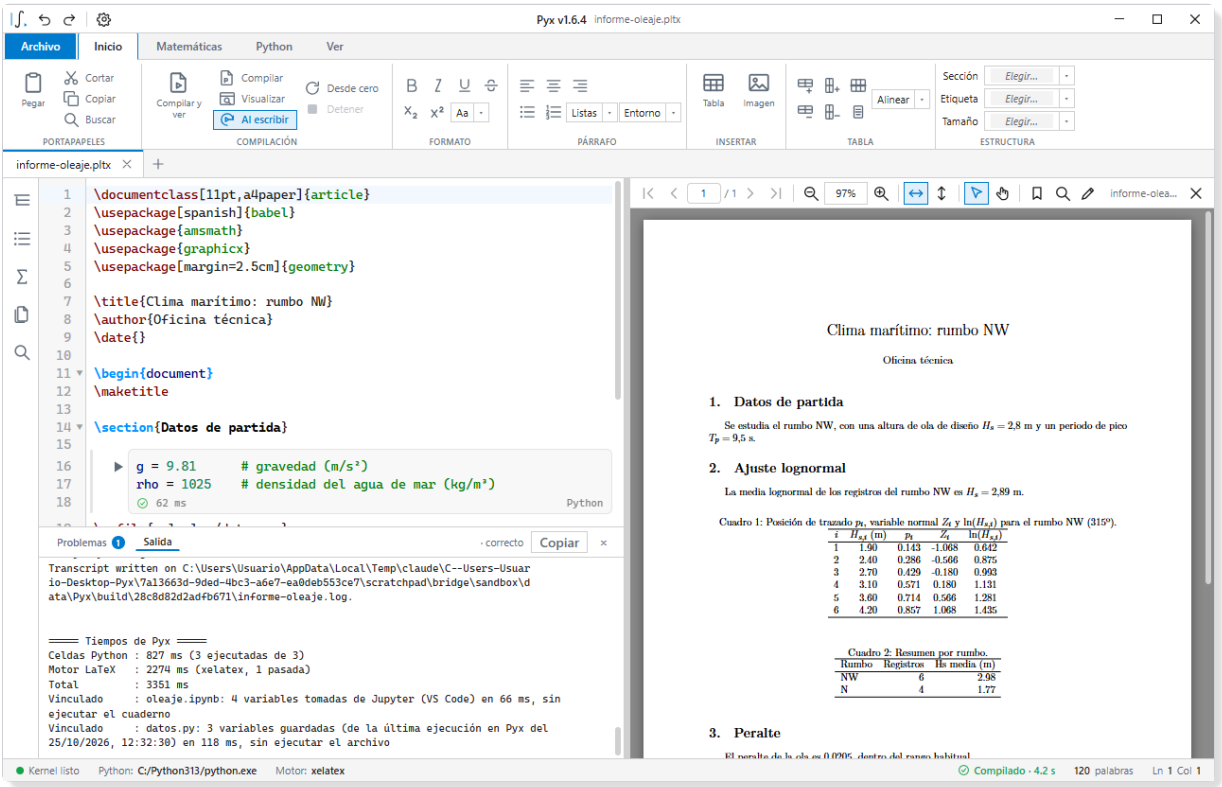


Figura 10.1: La salida de la compilación: al final, de dónde salieron los valores de cada archivo vinculado.

Nota Pyx compila una copia del documento (`.build.tex`) con los valores ya sustituidos, en una carpeta de trabajo fuera del proyecto. Los números de línea de los errores se traducen siempre a los del documento original.

11

El visor de PDF

El PDF se muestra a la derecha del editor y se actualiza solo después de cada compilación, conservando la página y el zoom.

11.1 La barra del visor

Navegación primera página, anterior, número de página (escribe uno para ir a él), siguiente y última.

Zoom alejar, acercar (o `Ctrl`+rueda del ratón) y el porcentaje actual, que vuelve al 100 % con un clic; ajustar al ancho o al alto de la página.

Herramientas seleccionar texto o mover la página con la mano (también con la rueda pulsada). Un doble clic sobre el PDF activa la lupa.

Marcadores el índice del PDF, para ir a cualquier sección (sección 11.3).

Buscar (`Ctrl`+`F` con el foco en el visor) busca en el texto del PDF; `Intro` y `Mayús`+`Intro` recorren los resultados.

Anotar lápiz, resaltador, línea, flecha, rectángulo, nota de texto y borrador, con colores y grosores. Las anotaciones se guardan con el documento y siguen al zoom.

Contraste oscurece el fondo de las páginas para trabajar de noche.

Ampliar oculta el editor y deja solo el visor.

Ventana auxiliar saca el visor a una ventana propia, ideal para un segundo monitor. Las compilaciones la siguen actualizando; al cerrarla, el visor vuelve a su sitio.

Guardar una copia del PDF y **abrir en la aplicación externa** (el lector de PDF del sistema).

11.2 Del código al PDF y del PDF al código (SyncTeX)

Del PDF al código: `Ctrl`+clic sobre una palabra del PDF lleva a la línea del código que la produce, con esa palabra seleccionada.

Del código al PDF: `Ctrl`+`Alt`+`F` (o `Ctrl`+clic en el editor) lleva al punto del PDF que corresponde a la línea del cursor y lo resalta un momento.

El PDF sigue lo que escribes: con la compilación automática, tras cada compilación el visor se desplaza a lo que estás editando si había quedado fuera de la vista (se puede desactivar en **Configuración** → **Visor PDF**).

Funciona también en proyectos de varios archivos y con Tectonic: Pyx lee directamente el índice de SyncTeX, sin depender de programas externos.

11.3 Marcadores

El botón de marcadores abre, a la izquierda del PDF, su índice (figura 11.1). Si el documento carga el paquete `hyperref`, se muestra el índice del propio PDF; si no, las secciones del documento, que Pyx lleva a su sitio en el PDF mediante SyncTeX. Un clic lleva a la sección.

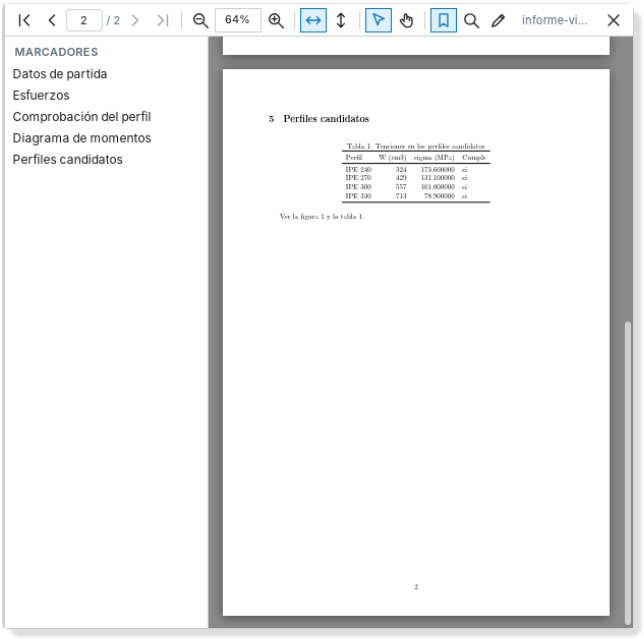


Figura 11.1: Los marcadores del PDF.

12

Macros en Python

Las macros automatizan tareas de edición repetitivas. En TeXstudio se escriben en JavaScript; en Pyx, en Python. Una macro son unas líneas de Python que leen:

`seleccion` el texto seleccionado en el editor;

`texto` el documento completo;

`linea` el número de la línea del cursor;

`variables` las variables calculadas por las celdas del documento (solo lectura);

y dejan en `resultado` el texto que sustituye a la selección. Si `resultado` se queda en `None`, el documento no cambia; lo que se imprima con `print()` se muestra en un aviso.

Pyx trae cinco macros de ejemplo: pasar a mayúsculas, ordenar líneas, convertir líneas en una lista `itemize`, insertar una tabla con las variables numéricas del documento y contar las palabras de la selección.



Figura 12.1: El editor de macros.

Ejecutar: desplegable Macro de la pestaña **Python**, o `Alt`+`Mayús`+`1` a `9` para las nueve primeras.

Crear y editar: Editar macros… en el mismo desplegable. Los cambios se guardan al momento; Ejemplos recupera las de fábrica.

Por ejemplo, esta macro convierte la selección «3,5» en «`\qty{3.5}{\metre}`»:

```
valor = seleccion.strip().replace(",", ".")
resultado = r"\qty{" + valor + r"}{\metre}"
```

Las macros se ejecutan en un espacio de nombres propio dentro del kernel: pueden usar cualquier librería instalada, pero no modifican las variables del documento.

13

Configuración

El engranaje de la barra de título abre **Configuración** (figura 13.1). El cuadro de búsqueda de arriba encuentra cualquier opción por su nombre. Los cambios se aplican al momento. Restablecer esta sección y Restablecer todo vuelven a los valores de fábrica.

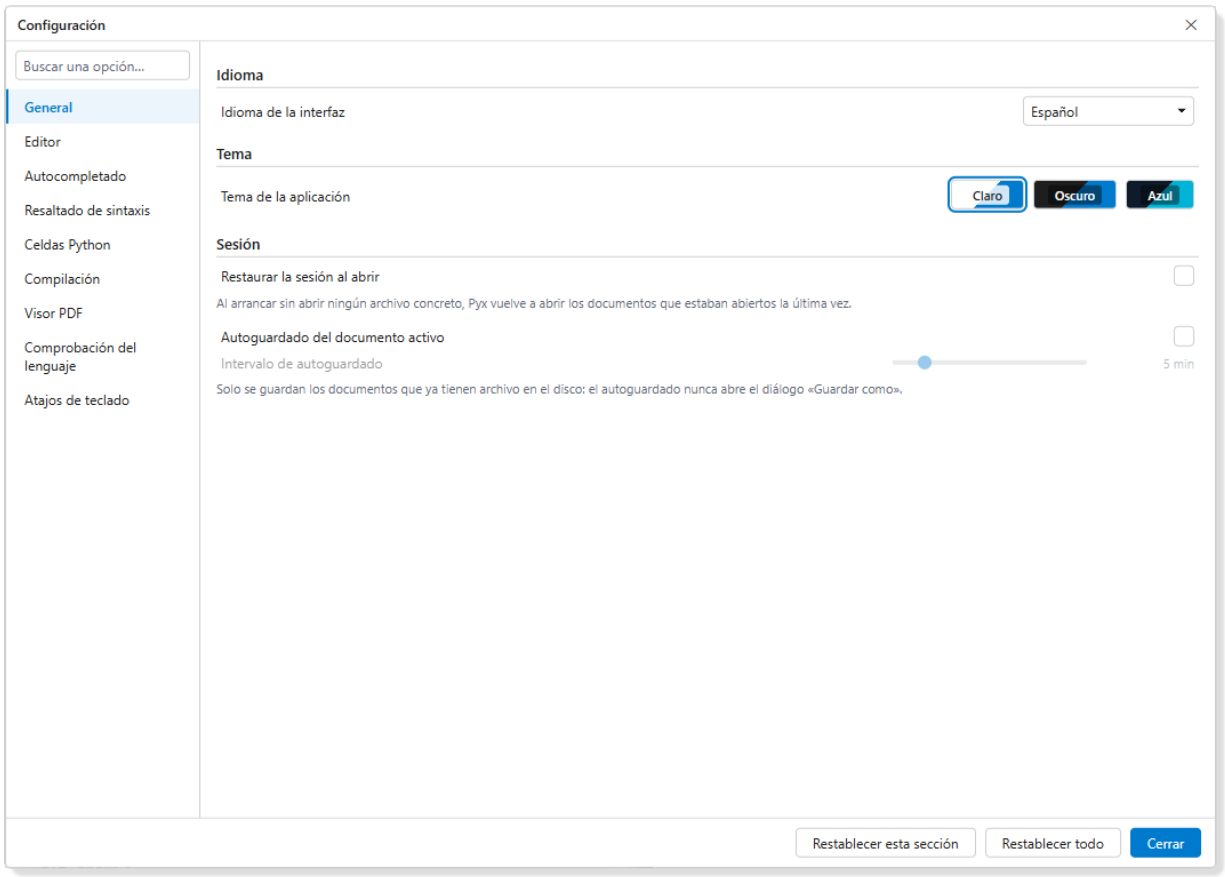


Figura 13.1: La categoría «General» de Configuración.

General idioma de la interfaz (español o inglés), tema y comportamiento de la sesión (reabrir los documentos al arrancar).

Editor tipografía (fuente, tamaño, interlineado), presentación (números de línea, minimapa, guías de sangría, línea activa, ajuste de línea), escritura (sangría automática, cierre de paréntesis, tabulación), espacios en blanco y desplazamiento, y las ayudas de LaTeX (vista previa de fórmulas, renombrar entornos, marcas del registro).

Autocompletado activarlo, mostrarlo al escribir, distinguir mayúsculas, seleccionar la primera opción e iconos.

Resaltado de sintaxis los colores del código LaTeX y de Python, con dos paletas para Python (la clásica y la de VS Code 2026).

Celdas Python el intérprete, el Python de Pyx, los valores en gris de `\py{}`, la ejecución reactiva y guardar los resultados dentro del documento.

Compilación compilación automática y su retardo, motor precalentado, comprobaciones antes de compilar (errores de LaTeX, ChkTeX), codificación de los archivos, motor y Tectonic.

Visor PDF ajuste inicial, contraste, la lupa y la sincronización (seguir el cursor, sincronizar la selección).

Comprobación del lenguaje corrector ortográfico e idioma, gramática y servidor de LanguageTool.

Atajos de teclado cambia cualquier atajo: pulsa Cambiar y teclea la combinación nueva. Los conflictos se marcan en rojo.

14

Atajos de teclado

Los atajos de la aplicación se pueden cambiar en **Configuración** → **Atajos de teclado**; estos son los de fábrica. En macOS, `Ctrl` es `Cmd`.

Atajo	Acción
Archivo	
`Ctrl` + `N`	Nuevo documento
`Ctrl` + `O`	Abrir
`Ctrl` + `S`	Guardar
`Ctrl` + `W`	Cerrar la pestaña
Edición	
`Ctrl` + `Z` / `Ctrl` + `Y`	Deshacer / rehacer
`Ctrl` + `F`	Buscar (en el editor o en el PDF, según el foco)
`Ctrl` + `Mayús` + `F`	Buscar en todo el proyecto
`Ctrl` + `B` / `Ctrl` + `I`	Negrita / cursiva
`Ctrl` + `T`	Comentar o descomentar
`Ctrl` + `A`	Seleccionar todo (dentro de una celda: su código)
`Ctrl` + `Espacio`	Mostrar el autocompletado
`Alt` + `↑` / `Alt` + `↓`	Mover la línea o la celda
`Alt` + `←` / `Alt` + `→`	Posición anterior / siguiente del cursor
`Ctrl` + `F2`	Poner o quitar un marcador
`F2` / `Mayús` + `F2`	Marcador siguiente / anterior
`F8` / `Mayús` + `F8`	Problema siguiente / anterior
`Ctrl` +clic en `\ref{}`	Ir a su `\label`
`Alt` + `Mayús` + `1` … `9`	Ejecutar una macro
Python	
`Ctrl` + `Intro`	Ejecutar la celda actual
`Mayús` + `Intro`	Ejecutar la celda y avanzar
`Ctrl` + `Alt` + `C`	Nueva celda
`Ctrl` + `Alt` + `Intro`	Ejecutar todas las celdas
`Ctrl` + `Alt` + `R`	Reiniciar el kernel
`Esc`	Cerrar la ayuda de una función
Compilación	
`Ctrl` + `Mayús` + `B`	Compilar y ver
`Ctrl` + `Alt` + `Mayús` + `B`	Recompilar desde cero
Vista	
`Ctrl` + `Alt` + `F`	Ir a esta línea en el PDF
`Ctrl` +clic en el PDF	Ir a ese punto del código

Atajo	Acción
`Ctrl` + `Alt` + `P`	Mostrar u ocultar el visor
`Ctrl` + `Alt` + `D`	Dividir el editor
`Ctrl` + `Alt` + `Z`	Modo zen
`Ctrl` + `Alt` + `T`	Terminal
`Ctrl` +rueda	Zoom del editor o del PDF

15

Solución de problemas

La barra de estado dice «Python: —» o el kernel está en rojo

Pyx no encuentra un intérprete de Python que funcione. Abre **Configuración** → **Celdas Python** → **Python de Pyx** → **Preparar o actualizar…** para que Pyx prepare el suyo, o elige un intérprete concreto en el desplegable **Python** de la pestaña **Python**.

«No se detectó LaTeX»

Instala MiKTeX (Windows), MacTeX (macOS) o TeX Live (Linux), o pulsa **Configuración** → **Compilación** → **Descargar Tectonic**. Si acabas de instalar una distribución, reinicia Pyx para que la detecte.

Una celda falla con `ModuleNotFoundError`

La librería no está instalada en el Python que usa Pyx. Pulsa Instalar … en la salida de la celda, o el aviso ! Falta de la barra de estado. Si usas un Python del sistema que no admite instalar paquetes (algunos Linux), haz que Pyx prepare el suyo.

`%%render` no funciona

Añade `import handcalcs.render` en una celda anterior. Si el aviso dice que falta IPython, instálalo (o prepara el Python de Pyx, que ya lo incluye).

Un `\py{}` sale vacío o con un error en el registro

La expresión no se puede evaluar: una variable que no existe (una celda anterior falló o no se ha ejecutado), un error de escritura o una excepción. El panel **Problemas** indica el motivo exacto de cada `\py{}`.

Un cuaderno vinculado se ejecuta otra vez

Pyx solo ejecuta un cuaderno o un archivo `.py` cuando no puede tomar sus valores: Jupyter no lo tiene abierto y no hay valores guardados de ese mismo código. El registro (**Salida**) dice el motivo en la línea «Vinculado». Lo más habitual: el código del cuaderno ha cambiado desde la última vez (guardar el cuaderno tras editar una celda cuenta como cambio). Abre el cuaderno en Jupyter o VS Code y ejecútalo allí: la siguiente compilación tomará los valores de ese kernel.

«Jupyter no tiene …» en los avisos

El documento usa una variable del cuaderno que su kernel de Jupyter todavía no ha calculado: ejecuta en Jupyter la celda que la define y vuelve a compilar.

El resultado no se ha actualizado tras cambiar un archivo de datos

Pyx vuelve a ejecutar una celda si el archivo que leyó con `open()` (o con pandas, NumPy, openpyxl…) ha cambiado. Si lo lees de otra forma, compila con Desde cero.

Una celda no termina

Pulsa en la celda o Interrumpir en la cinta. Si a los dos segundos no se ha detenido, el botón pasa a Forzar parada: termina el proceso de Python (se pierden las variables) y la siguiente ejecución empieza con un kernel nuevo.

La compilación tarda mucho

- Comprueba en el registro (**Salida**) si el tiempo se va en las celdas o en LaTeX.
- Si son las celdas: con la ejecución reactiva solo se repiten las afectadas; una celda lenta que se ejecuta siempre seguramente lee un archivo que cambia o usa algo que Pyx no puede analizar (`exec`, magias de IPython…).
- Si es LaTeX: comprueba que el motor precalentado está activo; las figuras muy grandes (PNG de varios megas) también ralentizan.

El PDF no se corresponde con lo que he escrito

La barra de estado indica si la última compilación terminó con errores. Si dudas, usa Recompilar desde cero, que no reutiliza nada de compilaciones anteriores.

Las ventanas, los menús o el texto se ven pequeños o borrosos

Ajusta el tamaño de la fuente del editor en **Configuración** → **Editor** → **Tipografía**, y el zoom del PDF con Ctrl+rueda.

16

Glosario

Archivo vinculado Un cuaderno de Jupyter o un archivo `.py` que el documento usa con `\pynotebook` o `\pyfile`: sus variables están disponibles desde ese punto sin copiar su código.

Celda Bloque de código Python dentro del documento, entre `%#python` y `%#end`.

Compilar Convertir el documento en PDF: ejecutar las celdas necesarias, sustituir los valores y ejecutar LaTeX.

Contenedor `.pltx` El formato de documento de Pyx: código, resultados de las celdas y archivos auxiliares en un solo archivo.

Early cutoff No volver a ejecutar las celdas que dependen de otra si esta ha dado el mismo resultado que antes.

Grafo de dependencias La relación entre las celdas según las variables que definen y usan; decide qué celdas hay que repetir tras un cambio.

jedi Biblioteca de autocompletado de Python que usan IPython, JupyterLab y Pyx.

Kernel El proceso de Python que ejecuta las celdas y conserva sus variables.

Motor precalentado Proceso de LaTeX con el preámbulo ya cargado, esperando la siguiente compilación.

SyncTeX Índice que relaciona cada punto del PDF con su línea de código.

Tectonic Motor de LaTeX autónomo que descarga los paquetes cuando hacen falta.

uv Instalador de Python y de paquetes, muy rápido, que usa Pyx para preparar su propio Python.

Créditos

Pyx se apoya en el trabajo de muchos proyectos de software libre. Entre ellos: Tauri, SolidJS, CodeMirror, PDF.js, KaTeX, DOMPurify, nspell y los diccionarios Hunspell (RLA para el español, SCOWL para el inglés), uv y python-build-standalone, Tectonic, jedi, handcalcs, y las ideas de TeXstudio, Jupyter y JupyterLab, marimo, MATLAB, Mathcad, CalcTree, texlab, latexmk, tex-fast-recompile y TeXpresso. El archivo `README.md` del código fuente detalla qué usa Pyx de cada uno.



Este manual se compiló con Pyx el 25 de octubre de 2026.
Python 3.13.7 · NumPy 2.3.4 · xelatex